REDROB HACKATHON · DATA & AI CHALLENGE

Ranking Candidates the Way a Great Recruiter Actually Would

A multi-signal candidate ranker built to beat keyword matching — not by using a bigger model, but by reading career-history substance, behavioral availability, and the gap between what's listed and what's actually demonstrated.

100,000
CANDIDATES RANKED

~80s
END-TO-END RUNTIME

0
HONEYPOTS IN TOP 100

CPU-only
NO NETWORK CALLS

THE PROBLEM

100,000 profiles. Most of them are a trap.

The JD asks for a system that ranks like a great recruiter — not a keyword matcher. The dataset is built to punish anyone who tries the easy way out.

~5,200

Keyword stuffers

Marketing Managers, Accountants, Mechanical Engineers with ‘RAG’, ‘Pinecone’, ‘LangChain’ listed as expert skills — at 0 months of actual use and 0 endorsements.

~10%

Title/substance mismatch

Current-role description doesn't match the stated title at all — read the career narrative, not the label, or you'll misjudge 1 in 10 profiles.

~30–80

Honeypots

Internally impossible timelines — e.g. ‘171 months’ (14+ years) at a role that started 21 months ago. Designed to catch systems that don't sanity-check the data.

~70%

Irrelevant roles, by design

The bulk of the pool is genuinely unrelated work — HR, Sales, Ops. The real signal is a thin slice of ~1,500–2,000 genuinely AI/ML-adjacent profiles.

ARCHITECTURE

Five signals, one composite score

No single signal is trusted alone. Semantic similarity reads meaning; structured features read facts; disqualifiers and honeypot checks gate the result; behavioral signals weight by reachability.

Semantic match

TF-IDF + SVD

Candidate free text vs. the JD's meaning-dense sections — cosine similarity in latent topic space.

Structured features

feature_extractor.py

Title bucket, trust-weighted skills, experience band, location/visa, notice period, title ↔ description coherence.

Disqualifiers

multiplicative

Services-only career, research-only, excluded domain, architecture drift, job-hopping, no-visa geography.

Honeypot gate

honeypot_detector.py

4 internal-consistency checks. 2+ flags → hard floor near zero.

Behavioral signal

behavioral_signals.py

Recency-decayed availability multiplier from 23 redrob_signals fields.

```
score = base_fit × disqualifier_mult × availability_mult   (honeypot → floor near 0)
```

WHY TF-IDF + SVD, NOT A NEURAL EMBEDDING MODEL

Fast, explainable, and it fits the budget

THE HARD BUDGET

- ≤ 5 minutes wall-clock for 100K candidates
- ≤ 16 GB RAM
- CPU only — no GPU
- Zero network calls during ranking

A hosted sentence-transformer call per candidate, at 100K candidates, blows this budget on latency alone — before counting the "zero network" rule that rules out hosted embedding APIs entirely.

MEASURED — FULL 100K RUN

Wall-clock time *1 vCPU sandbox*	**~80–140s**
Peak RAM *well under 16 GB cap*	**~2.8 GB**
Network calls *fully offline*	**0**
Validator result *validate_submission.py*	**PASS**

TF-IDF + truncated SVD (LSA) captures topic-level co-occurrence meaning ('recommendation system' + 'Spark' + 'production' clustering near 'ranking' + 'retrieval') without exact keyword overlap — and every dimension traces back to n-grams, so it's fully inspectable, not a black box.

THE JD'S OWN ACCEPTANCE TEST

Substance beats jargon — by construction

"A Tier 5 candidate may never say 'RAG' or 'Pinecone', but if their career history shows they built a recommendation system at a product company, they're a fit. A candidate with every AI keyword listed but a 'Marketing Manager' title is not — no matter how perfect the skill list looks."

Marketing Manager

Keyword-stuffer pattern

Skills: RAG, Pinecone, LangChain, Embeddings, Fine-tuning LLMs — all "expert"

Track record: 0 months duration, 0 endorsements on every one

Career history: 7 years running brand campaigns and budgets

SCORE: 0.0396 — buried near the bottom of 100K

Backend Engineer, Flipkart

Never says "RAG" once

Skills: Python, XGBoost, A/B Testing, Search Ranking — "advanced"

Track record: 40–60 months duration, 5–12 endorsements each

Career history: Owns the recommendation system end-to-end — candidate gen, ranking model, A/B harness

SCORE: 0.5365 — correctly ranked ahead

Verified in tests/test_scorer_end_to_end.py — re-run as a regression check, not a one-off demo.

HONEYPOT DETECTION

Four checks an internally-impossible profile can't pass

No ground-truth labels are available, so detection relies on internal-consistency checks a careful human reviewer would run — the same record can't tell two contradictory stories.

Experience vs. history

Stated years_of_experience vs. sum of career_history durations. Gap > 2.5 years → flagged.

Duration vs. dates

duration_months vs. actual start_date–end_date span. Example found: 171 months claimed at a role started 21 months ago.

Expert, zero track record

3+ skills marked ‘expert’ with 0 months' duration — you cannot be an expert in something used for zero time.

Education date inversion

A degree's end_year recorded before its start_year.

DECISION RULE

2+ independent flags → hard gate, score floored near zero. **1 flag** → mild soft penalty only — more likely benign data noise than a deliberate honeypot.

Result on the full 100,000-candidate pool: 0 honeypots reached the final top 100.

DISQUALIFIERS & AVAILABILITY

Multiplicative penalties, not additive nudges

NEAR-HARD "NO"s — scorer.py

Pure services-only career — `× 0.35`

TCS/Infosys/Wipro/etc. and nothing else

Research-only, no production — `× 0.30`

Academic-lab language, zero deploy/ship signal

Excluded domain, no NLP/IR — `× 0.40`

CV/speech/robotics-primary, no retrieval exposure

Architecture drift — `× 0.55`

18+ months in an architect/lead role, no recent code

Outside India, no sponsorship — `× 0.55`

JD doesn't sponsor visas — real practical hurdle

Frequent job-hopping — `var.`

Average tenure under 12–24 months across 3+ roles

Multiplicative so a real disqualifier can't be "outscored" by an unrelated strength — but disqualified candidates still differentiate among themselves near the bottom of the ranking.

AVAILABILITY MULTIPLIER — behavioral_signals.py

"A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% recruiter response rate is, for hiring purposes, not actually available." — job_description.md

`last_active_date`	exponential recency decay, 45-day half-life
`recruiter_response_rate`	highest single weight (30%)
`open_to_work_flag`	binary modifier
`interview_completion_rate`	secondary corroboration
`offer_acceptance_rate`	neutral default if no prior offers

Applied as a multiplier, never an additive bonus

REASONING COLUMN

Every claim traces to a real field. No LLM call.

Template-driven generation directly interpolates each candidate's own fields — there is no generative step that could invent a skill, employer, or number.

RANK #1 IN THE FINAL SUBMISSION

CAND_0064326 · score 0.8151

"Strong fit. Search Engineer at Sarvam AI, 7.6 yrs experience, based in Gurgaon, Haryana. Matches on vector database / hybrid search experience, strong Python background, embeddings/retrieval experience. 94% recruiter response rate, 45-day notice, last active 2026-05-21."

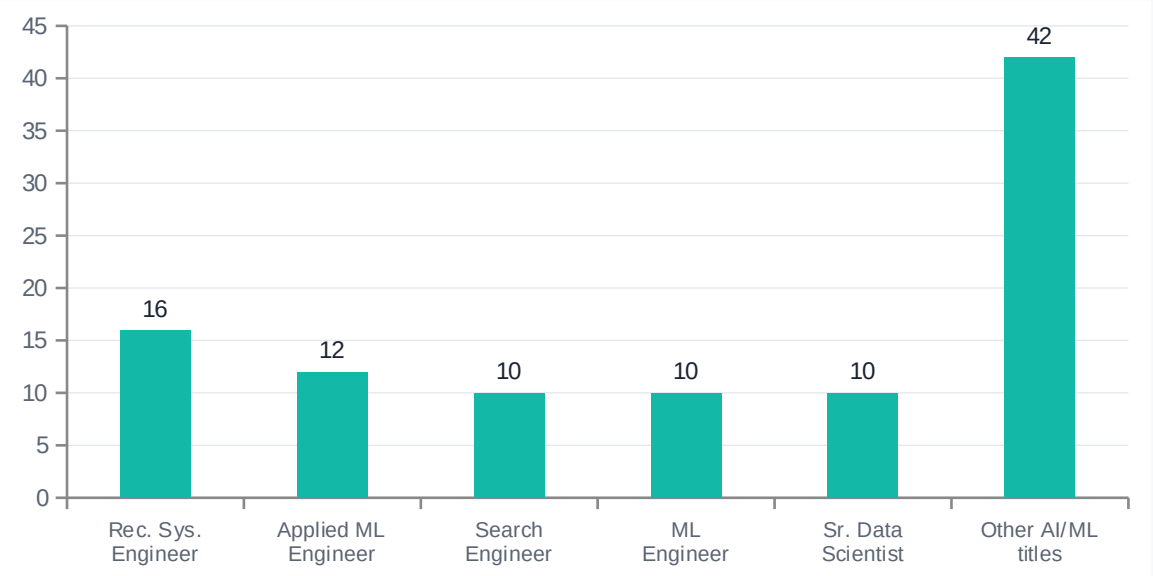
STAGE 4 MANUAL REVIEW CHECKLIST — submission_spec.md

- **Specific facts**
  Title, employer, years, named skills, exact % values — never vague praise
- **JD connection**
  Named must-have skill groups tie directly to the JD's own skills inventory
- **Honest concerns**
  Disqualifier flags are named in the sentence, even for top-ranked candidates
- **No hallucination**
  Every claim is interpolated from the candidate's own record, by construction
- **Variation**
  Each sentence pulls distinct fields (employer, %, skills) — no templated repeats
- **Rank consistency**
  Lead-in tone ('Strong fit' / 'Solid fit' / 'Reasonable fit') is set from the actual score band

VALIDATED AGAINST THE REAL DATASET

What actually landed in the top 100

TITLE DISTRIBUTION, TOP 100



100% genuine AI/ML-adjacent titles — 0 keyword-stuffers, 0 unrelated roles.

FULL-POOL VALIDATION, TOP 100

Honeypots included	**0 / 100**
Pure-services-only careers	**0 / 100**
India-based	**100 / 100**
In a JD-preferred city	**54 / 100**
Years of experience	**3.9 – 8.0 (avg 5.8)**
Avg. recruiter response rate	**75%**
validate_submission.py	**PASS**

HONEST TRADEOFFS

Where this system is intentionally simple

Every simplification below was a deliberate call under the compute/time budget — not an oversight. Listed here because we'd rather defend a known tradeoff than have it discovered at interview.

TF-IDF/SVD has no true paraphrase understanding

Co-occurrence statistics, not deep semantics — unusual phrasing for common work can score lower than it should. Mitigated by capping semantic similarity at ~30% of the composite weight.

Title ↔ description coherence is a heuristic, not a parser

Covers the dataset's common titles; falls back to neutral for unrecognized ones, and is weighted as a soft 0.65× multiplier rather than a hard exclusion specifically to bound false-positive risk.

Geography/visa penalty is a single blunt multiplier

The JD calls non-India location ‘case-by-case’ — a single multiplier can't capture every edge case (e.g. an Indian citizen briefly abroad). Simplified deliberately, not by accident.

REPRODUCE IT YOURSELF

One command. No network. Under two minutes.

```
$ pip install -r requirements.txt && python rank.py --candidates ./candidates.jsonl --out
./submission.csv
```

src/	jd_parser, semantic_matcher, feature_extractor, honeypot_detector, behavioral_signals, scorer, reasoning_generator
tests/	feature checks, honeypot checks, and the JD's own keyword-stuffer acceptance test as a regression
docs/	bundle reference docs converted to markdown for easy diffing
README.md	architecture, design rationale, reproduce command, honest limitations

9 commits, each one a real step — including a test that caught a real scoring bug before this submission.

Thank you — questions welcome at the Stage 5 interview.